

Practical 1.

1.

- a) **Installation: Configuration & Customisations of Linux**
- b) **Introduction to GCC compiler: Basics of GCC, Compilation of program, Execution of program**
- c) **Time stamping, Automating the execution using makefile**

a) Installation: Configuration & Customisations of Linux

Objective:

By the end of this guide, students will be able to successfully install and configure a Linux distribution on a VirtualBox virtual machine by following step-by-step procedures.

Specific Learning Outcomes:

- 1. **Prepare the Environment:** Download and install Oracle VM VirtualBox and a Linux ISO file (e.g., Ubuntu).
- 2. **Configure the Virtual Machine:** Create a new VM instance with appropriate hardware settings (allocated RAM, CPU cores, and virtual disk size).
- 3. **Execute the OS Installation:** Boot from the ISO image, complete the Linux setup wizard, and configure initial user credentials.
- 4. **Verify Functionality:** Launch the Linux VM, verify basic system settings, and enable VirtualBox Guest Additions for smooth screen scaling and clipboard integration.

Procedure:

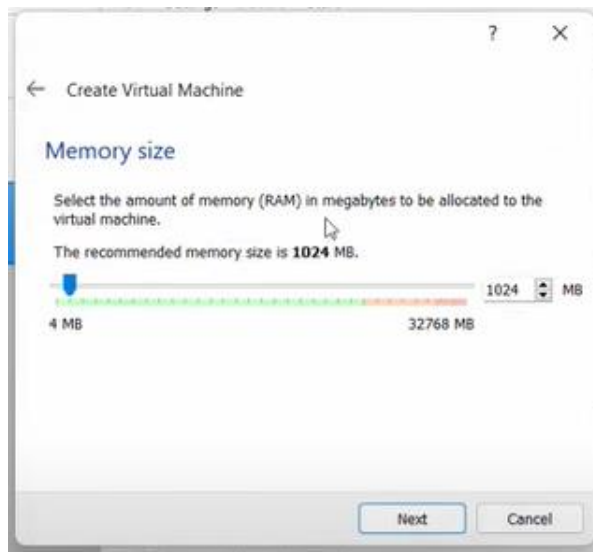
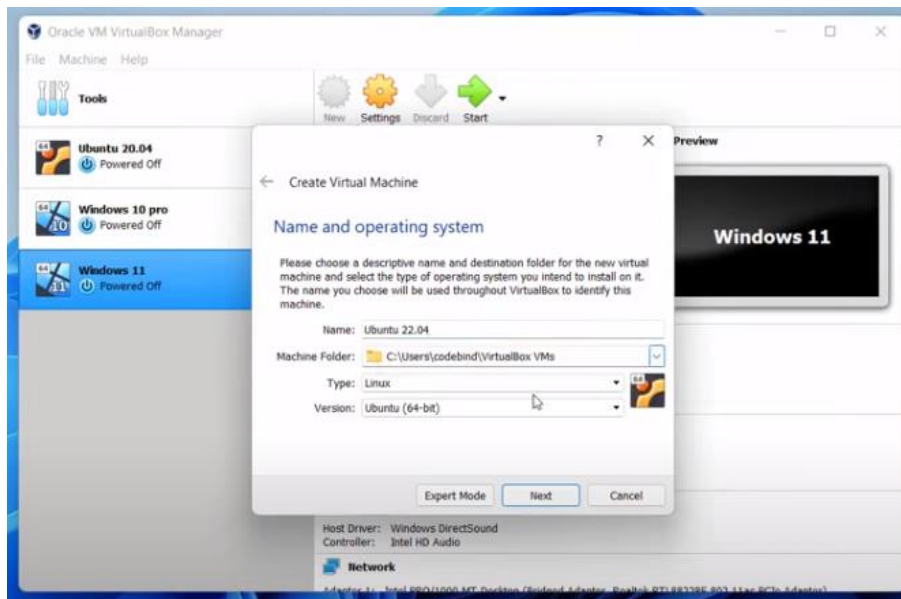
1. **Introduction to Linux :-**

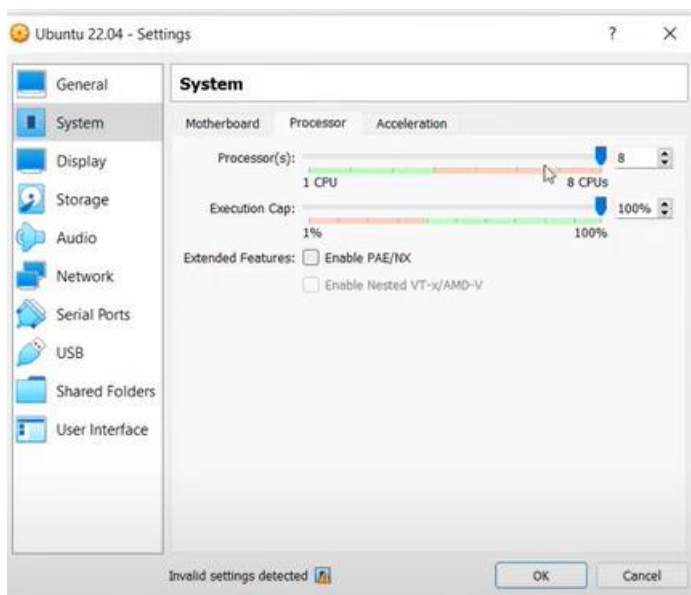
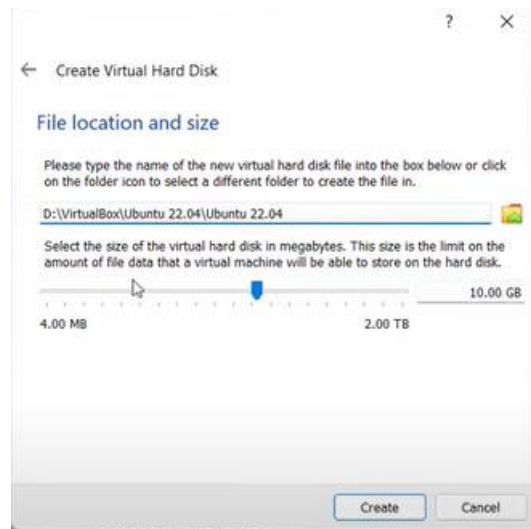
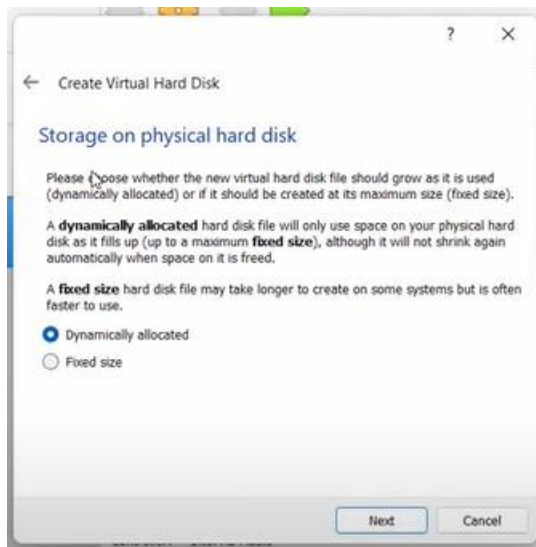
Linux is a Unix-like operating system kernel initially developed by Linus Torvalds in 1991. It is the core component of many different Linux distributions, which are complete operating systems built around the Linux kernel. Linux is open-source, meaning its source code is freely available for anyone to view, modify, and distribute under the terms of its respective licenses, such as the GNU General Public License (GPL). Due to its open nature, Linux has become widely adopted in various computing environments, including servers, desktops, embedded systems, mobile devices, and supercomputers. It is known for its stability, security, performance, and support for a wide range of hardware architectures.

Process for installation : -

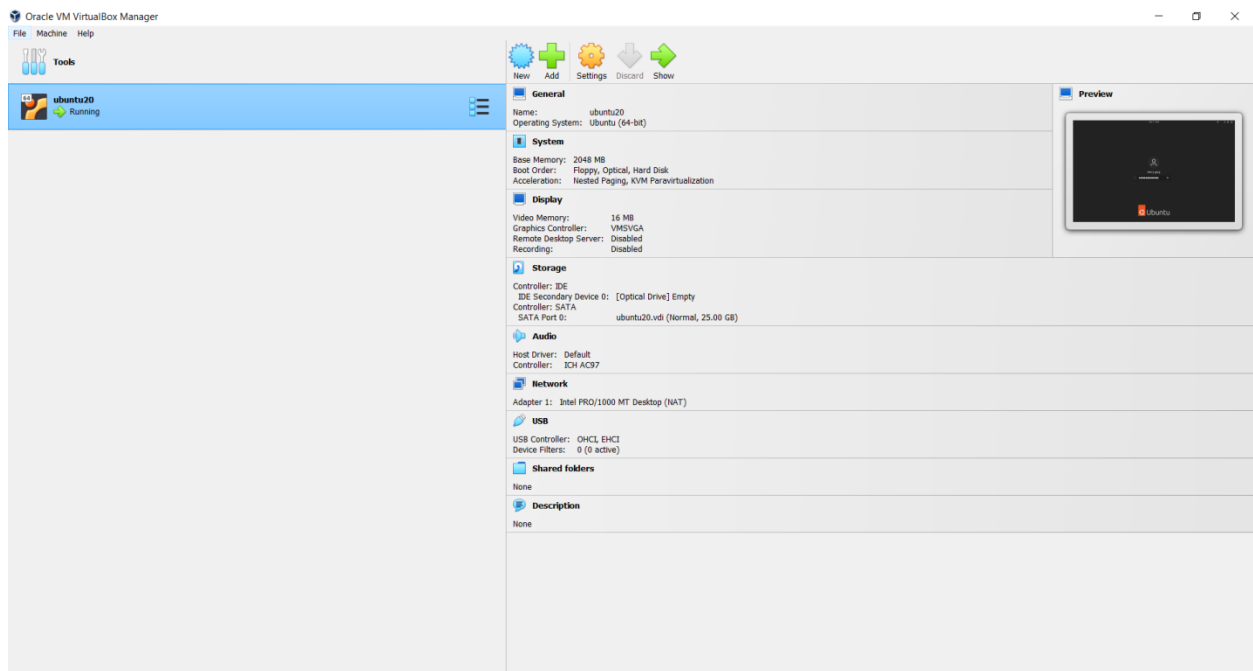
- 1.) Visit the official VirtualBox website (<https://www.virtualbox.org/>) and download the appropriate version for your operating system. Follow the installation instructions to install VirtualBox on your computer.
- 2.) Choose a Linux distribution you want to install. Popular choices include Ubuntu, Fedora, Debian, CentOS, etc. Download the ISO file of the distribution from its official website.
- 3.) Open VirtualBox and click on the "New" button. Open VirtualBox and click on the "New" button.
Select the type of operating system you are installing (e.g., Linux). Choose the version that matches your Linux distribution (e.g., Ubuntu 20.04). Click "Next".
- 4.) Choose the amount of RAM to allocate to the virtual machine. Ensure you allocate enough memory for smooth operation but leave enough for your host system. Click "Next".
- 5.) Select "Create a virtual hard disk now" and click "Create".
- 6.) Select the default option "VDI (VirtualBox Disk Image)" and click "Next".
- 7.) Choose either "Dynamically allocated" or "Fixed size" depending on your preference. "Dynamically allocated" will only use space on your physical disk as needed, while "Fixed size" will allocate all the space immediately. Click "Next".
- 8.) Choose the location where you want to store the virtual hard disk file and specify its size. Click "Create".
- 9.) Select the virtual machine you just created and click "Settings". Under "System", you can adjust the boot order and enable or disable features like EFI, IO APIC, etc. Under "Display", you can adjust video memory and enable 3D acceleration if needed. Under "Storage", select the empty disk and choose the Linux ISO file you downloaded earlier as the optical drive. Under "Storage", select the empty disk and choose the Linux ISO file you downloaded earlier as the optical drive.
- 10.) Start the virtual machine by clicking on the "Start" button. The Linux installation process should begin. Follow the on-screen instructions to install Linux. Make sure to select the appropriate options for language, keyboard layout, timezone, disk partitioning, etc. Once the installation is complete, eject the ISO image from the virtual optical drive and restart the virtual machine.

Configuration and Customization of Linux :-





Output:



b) Introduction to GCC compiler: Basics of GCC, Compilation of program, Execution of program

Objective: To provide students with a foundational understanding of the GNU Compiler Collection (GCC) toolchain, demystifying how human-readable C/C++ source code translates into architecture-specific machine binaries, and establishing core command-line proficiency required for system-level programming and debugging.

Learning outcome:

- By the end of this module, students will be able to:
- **Explain** the role and modular structure (Frontend, Middle-end, Backend) of the GCC compiler system.
- **Identify** and differentiate the four sequential stages of compilation (Preprocessing, Compilation, Assembly, and Linking).
- **Execute** terminal commands using key GCC flags (-E, -S, -c, -o, -Wall) to control build outputs.
- **Inspect** intermediate build files (.i, .s, .o) to diagnose translation issues and understand code transformation.
- **Run** generated binaries in a Linux/Unix environment and check process exit codes.

Procedure:

Basics of GCC compiler :-

The GCC (GNU Compiler Collection) is a suite of programming language compilers developed by the GNU Project. It is a free and open-source set of compilers that support various programming languages, including C, C++, Fortran, Ada, and others. The primary goal of GCC is to provide a high-performance, efficient, and portable compiler infrastructure for these languages.

Installation of GCC on Linux

By default, it comes with the most Linux distributions. We can verify it by executing the below command:

1. `gcc --version`
2. `g++ --version`
3. `make --version`

The above command will display the installed version of the GCC tool. If it is not installed, run the command **`sudo apt-get install gcc`**, or follow the below steps to install it:

Step 1: Update the package list:

To update the package list, execute the following command:

1. `sudo apt update`

```
ubuntu@ubuntu:~$ sudo apt update
gn:1 cdrom://Ubuntu 23.10.1 _Mantic Minotaur_ - Release amd64 (202310
it:2 cdrom://Ubuntu 23.10.1 _Mantic Minotaur_ - Release amd64 (202310
et:4 http://security.ubuntu.com/ubuntu mantic-security InRelease [109
et:5 http://archive.ubuntu.com/ubuntu mantic InRelease [256 kB]
et:6 http://archive.ubuntu.com/ubuntu mantic-updates InRelease [109 k
et:7 http://security.ubuntu.com/ubuntu mantic-security/main i386 Pack
et:8 http://archive.ubuntu.com/ubuntu mantic-backports InRelease [90.
et:9 http://security.ubuntu.com/ubuntu mantic-security/main amd64 Pac
et:10 http://archive.ubuntu.com/ubuntu mantic/main amd64 DEP-11 Metad
et:11 http://security.ubuntu.com/ubuntu mantic-security/main Translat
et:12 http://security.ubuntu.com/ubuntu mantic-security/main amd64 DE
et:13 http://security.ubuntu.com/ubuntu mantic-security/main DEP-11 4
et:14 http://security.ubuntu.com/ubuntu mantic-security/main DEP-11 6
et:15 http://archive.ubuntu.com/ubuntu mantic/universe amd64 DEP-11 M
et:16 http://security.ubuntu.com/ubuntu mantic-security/main DEP-11 6
et:17 http://security.ubuntu.com/ubuntu mantic-security/main amd64 c-
et:18 http://security.ubuntu.com/ubuntu mantic-security/restricted am
et:19 http://security.ubuntu.com/ubuntu mantic-security/restricted i3
et:20 http://security.ubuntu.com/ubuntu mantic-security/restricted Tr
et:21 http://archive.ubuntu.com/ubuntu mantic/multiverse amd64 DEP-11
et:22 http://security.ubuntu.com/ubuntu mantic-security/restricted am
et:23 http://archive.ubuntu.com/ubuntu mantic-updates/main i386 Packa
et:24 http://security.ubuntu.com/ubuntu mantic-security/universe amd6
et:25 http://archive.ubuntu.com/ubuntu mantic-updates/main amd64 Pack
et:26 http://archive.ubuntu.com/ubuntu mantic-updates/main Translatio
et:27 http://security.ubuntu.com/ubuntu mantic-security/universe i386
et:28 http://archive.ubuntu.com/ubuntu mantic-updates/main amd64 DEP-
et:29 http://security.ubuntu.com/ubuntu mantic-security/universe Tran
et:30 http://archive.ubuntu.com/ubuntu mantic-updates/main DEP-11 48x
et:31 http://security.ubuntu.com/ubuntu mantic-security/universe amd6
et:32 http://archive.ubuntu.com/ubuntu mantic-updates/main DEP-11 64x
et:33 http://security.ubuntu.com/ubuntu mantic-security/universe DEP-
et:34 http://archive.ubuntu.com/ubuntu mantic-updates/main DEP-11 64x
```

Step 2: Install the build-essential package:

It contains various packages such as gcc, g++, and make utility. Execute the below command to install it:

1. `sudo apt install build-essential`

```

... packages can be upgraded, run 'apt list --upgradable' to see them
ubuntu@ubuntu:~$ sudo apt install build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu dpkg-dev fakeroot
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libbinutils
  libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libfakeroot lib
  libnspr-dev libquadmath0 libsframe1 libstdc++-13-dev libtirpc-dev lib
  lto-disabled-list make manpages-dev rpcsvc-proto
Suggested packages:
  binutils-doc gprofng-gui debian-keyring g++-multilib g++-13-multilib
  flex bison gcc-doc gcc-13-multilib gcc-13-locales glibc-doc libstdc++
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential
  libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge
  libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libfakeroot lib
  libnspr-dev libquadmath0 libsframe1 libstdc++-13-dev libtirpc-dev lib
  manpages-dev rpcsvc-proto
The following packages will be upgraded:
  libc-bin libc6 libc6-dbg locales
4 upgraded, 40 newly installed, 0 to remove and 234 not upgraded.
Need to get 25.0 MB/81.2 MB of archives.

```

Step 3: Verify the installation:

To verify the installation, execute the `gcc -version` command as follows:

1. `gcc --version`

```

...processing triggers for libc-bin (2.38-1ubuntu1) ...
ubuntu@ubuntu:~$ gcc --version
gcc (Ubuntu 13.2.0-4ubuntu3) 13.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is
no warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR

```

Compilation of program:-

```
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
ubuntu@ubuntu:~$ mkdir g4
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
ubuntu@ubuntu:~$ cd g4
ubuntu@ubuntu:~/g4$ ls
ubuntu@ubuntu:~/g4$ touch g4.txt
ubuntu@ubuntu:~/g4$ ls
g4.txt
ubuntu@ubuntu:~/g4$ cat g4.txt
ubuntu@ubuntu:~/g4$ nano g4.txt
ubuntu@ubuntu:~/g4$ cat g4.txt
hello g4
ubuntu@ubuntu:~/g4$ ls -lu g4.txt
-rw-rw-r-- 1 ubuntu ubuntu 9 Feb 12 17:18 g4.txt
ubuntu@ubuntu:~/g4$ ls -l
total 4
-rw-rw-r-- 1 ubuntu ubuntu 9 Feb 12 17:18 g4.txt
ubuntu@ubuntu:~/g4$ ls -lc
total 4
-rw-rw-r-- 1 ubuntu ubuntu 9 Feb 12 17:18 g4.txt
ubuntu@ubuntu:~/g4$ stat g4.txt
  File: g4.txt
  Size: 9          Blocks: 8          IO Block: 4096   regular file
Device: 0,28      Inode: 15517         Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2024-02-12 17:18:37.010615514 +0000
Modify: 2024-02-12 17:18:25.036660105 +0000
Change: 2024-02-12 17:18:25.036660105 +0000
 Birth: 2024-02-12 17:17:15.093247154 +0000
ubuntu@ubuntu:~/g4$
```

```
  Birth: 2024-02-12 17:17:15.093247154 +0000
ubuntu@ubuntu:~/g4$ touch -a -t 200001181025.16 g4.txt
ubuntu@ubuntu:~/g4$ stat g4.txt
  File: g4.txt
  Size: 9          Blocks: 8          IO Block: 4096   regular file
Device: 0,28      Inode: 15517         Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2000-01-18 10:25:16.000000000 +0000
Modify: 2024-02-12 17:18:25.036660105 +0000
Change: 2024-02-12 17:32:34.611461236 +0000
 Birth: 2024-02-12 17:17:15.093247154 +0000
ubuntu@ubuntu:~/g4$ touch -m -t 200001181025.16 g4.txt
ubuntu@ubuntu:~/g4$ stat g4.txt
  File: g4.txt
  Size: 9          Blocks: 8          IO Block: 4096   regular file
Device: 0,28      Inode: 15517         Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2000-01-18 10:25:16.000000000 +0000
Modify: 2000-01-18 10:25:16.000000000 +0000
Change: 2024-02-12 17:33:22.302269442 +0000
 Birth: 2024-02-12 17:17:15.093247154 +0000
ubuntu@ubuntu:~/g4$ rm g4.txt
ubuntu@ubuntu:~/g4$ ls
ubuntu@ubuntu:~/g4$ cd
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
ubuntu@ubuntu:~$ cd
ubuntu@ubuntu:~$ rmdir g4
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
ubuntu@ubuntu:~$
```

Run first C program by gcc.

Execution of program:

Create a basic c program "Hello world!".

Create a file 'hello.c' and put below code in it:

```
1.#include<stdio.h>
```

```
void main() {
```

```
    printf("Hello");
```

```
}
```

A terminal window with a dark purple background and light green text. The commands and their outputs are as follows:
ubuntu@ubuntu:~\$ touch hello.c
ubuntu@ubuntu:~\$ vi hello.c
ubuntu@ubuntu:~\$ cat hello.c
#include<stdio.h>
void main()
{
printf("hello");
}

ubuntu@ubuntu:~\$ gcc hello.c
ubuntu@ubuntu:~\$ ls
Desktop Documents Downloads Music Pictures Public Templates Video
ubuntu@ubuntu:~\$./a.out
helloubuntu@ubuntu:~\$

c) Automating the execution using Make file.

Make first file named main.c and write the following:

```
#include<stdio.h>
```

```
#include "calc.h"
```

```
void main()
```

```
{
```

```
int a,b,c;
```

```
printf("Welcome to main\n");
```

```
a = add(3,4);  
printf("sum is %d",a);  
b = sub(3,2);  
printf("sub is %d",b);  
c = mul(3,2);  
printf("mul is %d",c);  
}
```

Second file named add.c and write

```
#include<stdio.h>  
#include "calc.h"  
int add(int a,int b)  
{  
    return a+b;  
}
```

Third file named sub.c and write

```
#include<stdio.h>  
#include "calc.h"  
int sub(int a,int b)  
{  
    return a-b;  
}
```

Fourth file named mul.c and write

```
#include<stdio.h>

#include "calc.h"

int mul(int a,int b)

{

return a*b;

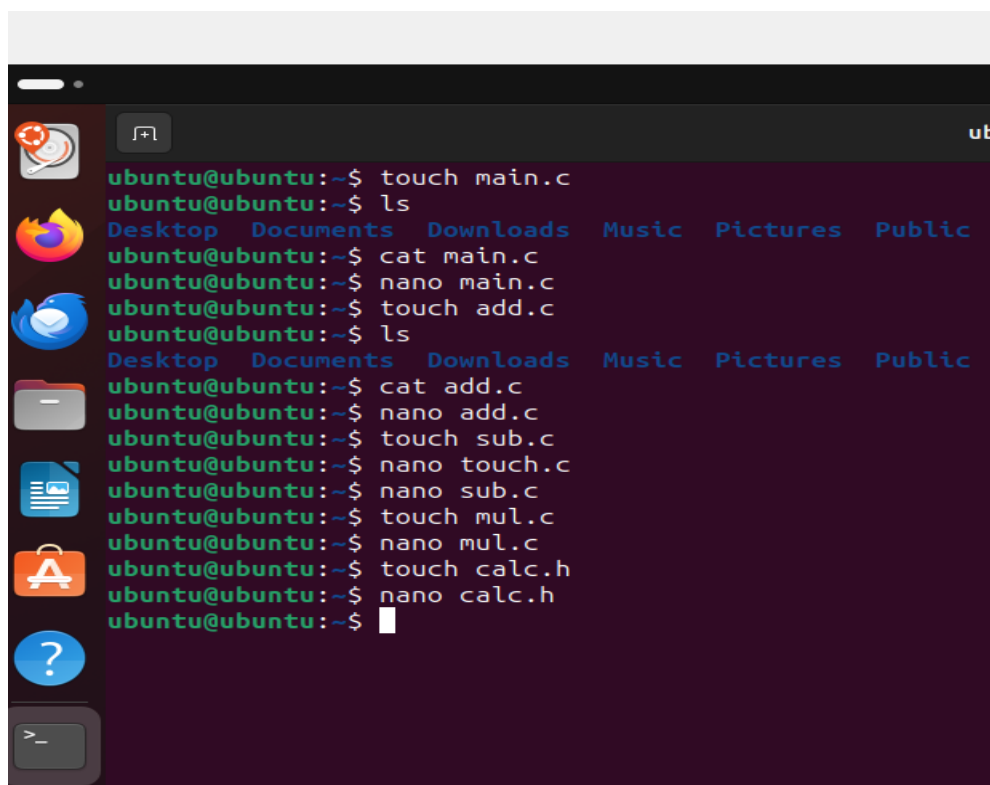
}
```

Fifth file named calc.h and write

```
int add(int ,int);

int sub(int,int);

int mul(int,int);
```

A terminal window on an Ubuntu system showing a series of commands to create and edit files. The user starts by creating main.c, then lists the directory contents. They then create add.c, list the directory again, and create sub.c. Next, they create mul.c and then calc.h. The terminal output shows the following sequence of commands and their results:

```
ubuntu@ubuntu:~$ touch main.c
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public
ubuntu@ubuntu:~$ cat main.c
ubuntu@ubuntu:~$ nano main.c
ubuntu@ubuntu:~$ touch add.c
ubuntu@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public
ubuntu@ubuntu:~$ cat add.c
ubuntu@ubuntu:~$ nano add.c
ubuntu@ubuntu:~$ touch sub.c
ubuntu@ubuntu:~$ nano touch.c
ubuntu@ubuntu:~$ nano sub.c
ubuntu@ubuntu:~$ touch mul.c
ubuntu@ubuntu:~$ nano mul.c
ubuntu@ubuntu:~$ touch calc.h
ubuntu@ubuntu:~$ nano calc.h
ubuntu@ubuntu:~$
```

Then create a file named **makefile** :

```
test.exe: main.o add.o sub.o mul.o
```

```
    gcc -o test.exe main.o add.o sub.o mul.o
```

```
main.o: main.c calc.h
```

```
    gcc -c main.c
```

```
add.o: add.c calc.h
```

```
    gcc -c add.c
```

```
sub.o: sub.c calc.h
```

```
    gcc -c sub.c
```

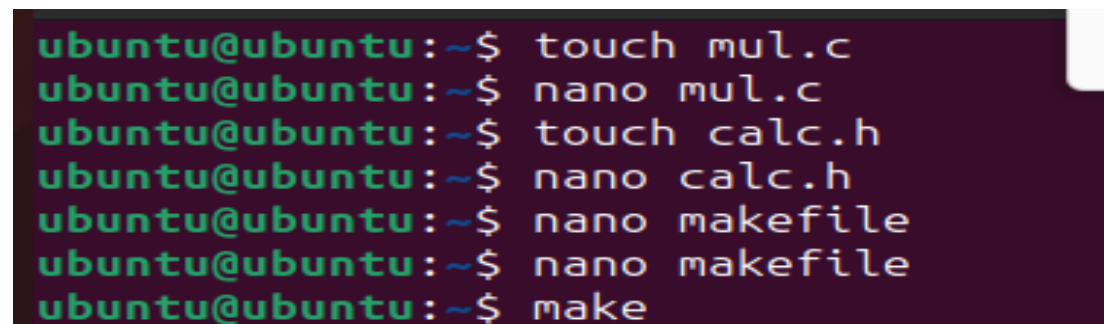
```
mul.o: mul.c calc.h
```

```
    gcc -c mul.c
```

Type: **./test.exe** To see the result

Then write clean:

```
rm *.O *.exe
```

A terminal window with a dark background and light green text. It shows a series of commands being entered at the prompt 'ubuntu@ubuntu:~\$'. The commands are: 'touch mul.c', 'nano mul.c', 'touch calc.h', 'nano calc.h', 'nano makefile', 'nano makefile', and 'make'.

```
ubuntu@ubuntu:~$ touch mul.c
ubuntu@ubuntu:~$ nano mul.c
ubuntu@ubuntu:~$ touch calc.h
ubuntu@ubuntu:~$ nano calc.h
ubuntu@ubuntu:~$ nano makefile
ubuntu@ubuntu:~$ nano makefile
ubuntu@ubuntu:~$ make
```

This is a Makefile used to compile a C program that is divided into multiple files. Let's understand it very simply.

1. What is the purpose?

Suppose your project has these files:

main.c

add.c

sub.c

mul.c

calc.h

The program performs addition, subtraction, and multiplication.

Instead of manually compiling every file, we use a Makefile. The make command checks which files have changed and compiles only what is necessary.

2. First rule

test.exe: main.o add.o sub.o mul.o

gcc -o test.exe main.o add.o sub.o mul.o

gcc -o test.exe main.o add.o sub.o mul.o

means:

- gcc → use the GCC compiler
- -o test.exe → name the output file test.exe
- main.o add.o sub.o mul.o → combine these object files

So, the final executable is test.exe.

3. Second rule

main.o: main.c calc.h

gcc -c main.c

The command:

`gcc -c main.c`

compiles `main.c` into:

`main.o`

Here, `-c` means compile only; don't create the final executable yet.

4. Third rule

`add.o: add.c calc.h`

`gcc -c add.c`

Result:

`add.c → gcc -c → add.o`

`calc.h` is listed as a dependency because `add.c` may use declarations from that header file.

5. Fourth rule

`sub.o: sub.c calc.h`

`gcc -c sub.c`

This creates:

`sub.c → gcc -c → sub.o`

6. Fifth rule

`mul.o: mul.c calc.h`

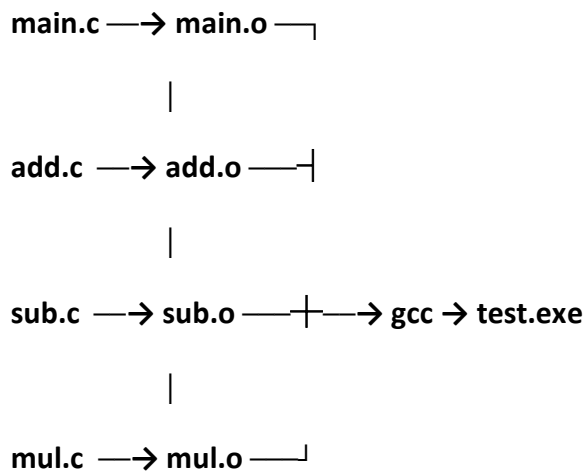
`gcc -c mul.c`

This creates:

`mul.c → gcc -c → mul.o`

Overall process

The Makefile works like this:



Why use a Makefile?

Imagine you have 100 C files and you change only `add.c`.

Without make, you might have to compile everything again.

With a Makefile, you simply run:

`make`

make checks timestamps and determines which files are newer.

For example:

`add.c` changed

↓

`add.o` is old

↓

compile `add.c` → `add.o`

↓

link object files

↓

create new `test.exe`

It avoids unnecessary compilation, saving time.

Important commands to remember

Command	Meaning
<code>gcc -c main.c</code>	Compile main.c into main.o
<code>gcc -c add.c</code>	Compile add.c into add.o
<code>gcc -c sub.c</code>	Compile sub.c into sub.o
<code>gcc -c mul.c</code>	Compile mul.c into mul.o
<code>gcc -o test.exe main.o add.o sub.o mul.o</code>	Combine object files to create test.exe
<code>make</code>	Automatically build the required files

In one sentence:
 .c files are first converted into .o object files, and then all .o files are combined by GCC to create the final test.exe.